Step 18 - Timer Initialization Flow 1
Reference Note 1
Learning Objectives 1
1. Initialization Overview 1
2. Discover Counter Frequency 1
3. Calculate Desired Interval 1
4. Read Current Counter 1
5. Program Using CVAL 1
6. Program Using TVAL 1
7. Enable Timer 1
Enable Configuration 1
8. Enable PPI 1
9. Enable CPU Interrupts 1
10. Complete Initialization Flow 1
Physical Timer Example 1
11. Expiry Sequence 1
Result 1
12. Periodic Timer Example 1
Recommended Method 1
13. End-to-End Timer Path 1
Common Initialization Mistakes 1
Boot-Time Checklist 1
Architecture Summary 1
Key Takeaways 1
Knowledge Check 1
Q1 1
Q2 1
Q3 1
Q4 1

Q5 1
Answer Key 1

Step 18 - Timer Initialization Flow

Reference Note

This chapter is based on the ARM Generic Timer Architecture documentation.

Previous chapters explained:

- System Counter
- Physical Counter
- Virtual Counter
- Physical Timer
- Virtual Timer
- Timer Expiry Logic
- GIC Interaction

This chapter brings everything together by showing:

How does software configure a timer?
What steps are required before the first timer interrupt occurs?
What happens from initialization to ISR execution?

Learning Objectives

After completing this chapter, you should understand:

- Basic timer initialization sequence
- Role of CNTFRQ_EL0
- Reading the current counter
- Programming TVAL and CVAL

- Enabling a timer
- Enabling the corresponding PPI
- Complete interrupt flow

1. Initialization Overview

Before a timer can generate interrupts:

Step	Action
1	Determine counter frequency
2	Read current counter value
3	Program timer deadline
4	Enable timer
5	Enable timer PPI
6	Enable CPU interrupts
7	Wait for expiry

2. Discover Counter Frequency

The first requirement is knowing:

How fast does the counter increment?

This information comes from:

CNTFRQ_EL0

Example:

Register	Value
CNTFRQ_EL0	24,000,000

Meaning:

24 million ticks per second

3. Calculate Desired Interval

Suppose the goal is:

Generate an interrupt every 1 ms

Using:

Frequency = 24 MHz

Calculation:

Ticks
=
Frequency × Time
=
24,000,000 × 0.001
=
24,000

Result:

Interval	Ticks
1 ms	24,000

4. Read Current Counter

Next read the current count.

Example:

MRS X0, CNTPCT_EL0

Suppose:

Register	Value
CNTPCT_EL0	1,000,000

This becomes the reference point.

5. Program Using CVAL

Absolute programming:

CVAL = CurrentCounter + Period

Example:

Item	Value
Current Count	1,000,000
Period	24,000
CVAL	1,024,000

Program:

MSR CNTP_CVAL_EL0, xN

6. Program Using TVAL

Relative programming:

TVAL = Period

Example:

Item	Value
TVAL	24,000

Program:

MSR CNTP_TVAL_EL0, xN

Hardware internally computes:

CVAL = CurrentCounter + TVAL

7. Enable Timer

Configure:

CNTP_CTL_EL0

Important control bits:

Bit	Purpose
ENABLE	Start timer
IMASK	Interrupt masking
ISTATUS	Expiry status

Enable Configuration

Field	Value
ENABLE	1
IMASK	0

Meaning:

Timer Running
Interrupt Delivery Enabled

8. Enable PPI

Timer expiry alone is insufficient.

The timer interrupt source must also be enabled.

Example:

Timer	Typical PPI
Physical Timer	30
Virtual Timer	27

Hypervisor Timer	26

The PPI must be enabled in the GIC.

9. Enable CPU Interrupts

The processor must also accept interrupts.

Requirement	Status
Timer Enabled	Yes
PPI Enabled	Yes
GIC Enabled	Yes
CPU IRQ Enabled	Yes

If any are missing:

No ISR executes

10. Complete Initialization Flow

Physical Timer Example

Step	Operation
1	Read CNTFRQ_EL0
2	Calculate period ticks
3	Read CNTPCT_EL0
4	Compute deadline
5	Program CNTP_CVAL_EL0
6	Enable CNTP_CTL_EL0
7	Enable PPI 30
8	Enable IRQs
9	Wait for expiry

11. Expiry Sequence

Suppose:

Counter	Value
Current Count	1,024,000
CVAL	1,024,000

Condition:

Current Count >= CVAL

becomes true.

Result

Step	Action
1	Timer expires
2	ISTATUS becomes 1
3	PPI asserted
4	GIC marks interrupt pending
5	CPU accepts interrupt
6	ISR executes

12. Periodic Timer Example

Inside the ISR:

Step	Action
1	Handle timer event
2	Compute next deadline
3	Program new CVAL
4	Return from interrupt

Recommended Method

Use:

NextDeadline += Period

rather than:

CurrentCounter + Period

This minimizes timing drift, as discussed in Step 8.

13. End-to-End Timer Path

Stage	Component
Time Source	System Counter
Time Read	Physical Counter
Compare Logic	Physical Timer
Interrupt Source	PPI
Interrupt Management	GIC
Software Execution	ISR

Common Initialization Mistakes

Mistake	Result
ENABLE = 0	Timer never runs
IMASK = 1	No interrupt delivery
PPI disabled	No interrupt seen
IRQ disabled	No ISR execution
Wrong tick calculation	Incorrect timer period
CVAL already expired	Immediate interrupt

Boot-Time Checklist

Item
CNTFRQ_EL0 read
Tick calculation correct
Deadline programmed
ENABLE = 1
IMASK = 0
PPI enabled
GIC enabled
CPU IRQ enabled

Architecture Summary

Component	Responsibility
CNTFRQ_EL0	Counter frequency
CNTPCT_EL0	Current count
CVAL/TVAL	Timer deadline
CNTP_CTL_EL0	Timer control
PPI	Interrupt source
GIC	Interrupt delivery
ISR	Software execution

Key Takeaways

Concept	Key Point
First Step	Read CNTFRQ_EL0
Timer Programming	Use CVAL or TVAL
Enable Bit	Starts timer

IMASK	Controls interrupt output
PPI	Must be enabled
GIC	Delivers interrupt
ISR	Final result of expiry

Knowledge Check

Q1

What register provides the counter frequency?

Q2

Why must timer intervals be converted into ticks?

Q3

What does ENABLE control?

Q4

What additional configuration is required after enabling a timer?

Q5

Does timer expiry guarantee ISR execution?

Answer Key

Question	Answer
Q1	CNTFRQ_EL0
Q2	Timers operate on counter ticks
Q3	Whether the timer runs
Q4	Enable PPI, GIC, and CPU interrupts

Q5	No, interrupt delivery must also succeed